

**SIMATS ENGINEERING**  
**CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS**

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA1709

|                        |                      |
|------------------------|----------------------|
| <b>Name</b>            | <b>192311097</b>     |
| <b>Register Number</b> | <b>Dillilokesh D</b> |

**COURSE OUTCOME COVERED**

**CO4:** Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

**Time Duration: 45 Minutes**

**QUESTIONS: 2**

**Total Marks:20**

**Code of Conduct:**

*I Dillilokesh D 192311097(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

**Experiment 1: Decision Tree Classification**

**Objective:** Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a)** Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b)** Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c)** Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

**Experiment 2: Reinforcement Learning – Q-Learning**

**Objective:** Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a)** Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters ( $\alpha$ ,  $\gamma$ ,  $\epsilon$ ).
- (b)** Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c)** Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

**RUBRICS FOR CALCULATION:**

| <b>Criteria</b>           | <b>Weightage</b> | <b>Level 4 – Exemplary</b>                                                                    | <b>Level 3 – Proficient</b>                                 | <b>Level 2 – Developing</b>                         | <b>Level 1 – Beginning</b>                       |
|---------------------------|------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25               | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30               | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20               | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15               | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10               | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |

# Experiment 1: Decision Tree Classification

## Aim

To build a Decision Tree classifier on the Iris dataset using Python and scikit-learn, and to understand how the tree actually decides where to split at each node (Gini index in our case), then check how good it is at telling the three iris species apart.

## Software Requirements

Since this is a software experiment rather than a hardware one, here's what I used instead of the usual apparatus list:

- A laptop/PC with Python 3.x installed
- Libraries: pandas, numpy, scikit-learn, matplotlib
- Jupyter Notebook (I like being able to re-run individual cells while debugging)
- The Iris dataset — didn't need to download anything separately since it ships directly with sklearn.datasets

## Theory / Working Principle

A decision tree is basically a flowchart that the algorithm builds for itself. At every node it asks a yes/no style question about one of the features (like 'is petal length  $\leq 2.45$ ?) and depending on the answer, the data gets routed left or right. It keeps doing this, splitting the data into smaller and smaller, purer groups, until each leaf mostly contains one class.

The tricky part is: how does it decide which question to ask at each node? That's where the Gini index comes in. Gini impurity basically measures how 'mixed up' a node is — a node with only one class has Gini = 0 (completely pure), while a node with classes evenly split has a high Gini value. At every possible split, the algorithm calculates the impurity of the resulting children and picks whichever feature/threshold combo drops the impurity the most. It's a greedy approach — it doesn't think ahead, it just takes whatever looks best right now, which is also why trees can sometimes overfit if you let them grow without limits.

$$\text{Gini}(\text{node}) = 1 - \sum (p_i)^2 \quad \text{where } p_i = \text{proportion of class } i \text{ in that node}$$

## Procedure

This is roughly how I worked through it, step by step:

1. Imported pandas, numpy, sklearn's tree/model\_selection/metrics modules, and matplotlib for plotting.
2. Loaded the Iris dataset using load\_iris() and put it into a pandas DataFrame so it's easier to look at.
3. Printed df.head() and df.dtypes to get a first look at the data — 4 numeric feature columns and the species label.
4. Ran df.isnull().sum() to check for missing values. Iris came back completely clean, 0 missing anywhere, so no imputation was needed here (though I still wrote a fallback in case there were any).
5. Label-encoded the species column (setosa=0, versicolor=1, virginica=2) since the tree needs numeric targets to train on.
6. Split the data 80/20 into train and test sets using train\_test\_split, with stratify=y so each species stays proportionally represented in both sets.
7. Created a DecisionTreeClassifier with criterion='gini' and trained it using .fit().
8. Used export\_text() to print the tree as readable if/else style rules, and plot\_tree() to get the visual version.
9. Predicted on the test set and compared predictions against the actual labels.

10. Computed accuracy, plotted a confusion matrix, and printed the full classification report.
11. Filtered the test results to pull out exactly which rows the model got wrong.

## Observations and Data Recording

First 5 rows of the raw dataset, just to confirm everything loaded properly:

| Sepal L | Sepal W | Petal L | Petal W | Species |
|---------|---------|---------|---------|---------|
| 5.1     | 3.5     | 1.4     | 0.2     | setosa  |
| 4.9     | 3.0     | 1.4     | 0.2     | setosa  |
| 4.7     | 3.2     | 1.3     | 0.2     | setosa  |
| 4.6     | 3.1     | 1.5     | 0.2     | setosa  |
| 5.0     | 3.6     | 1.4     | 0.2     | setosa  |

A sample of the test-set predictions I recorded (full 30 rows were checked, showing a representative slice here including both mistakes the model made):

| Sepal L | Sepal W | Petal L | Petal W | Actual     | Predicted  | Match? |
|---------|---------|---------|---------|------------|------------|--------|
| 4.4     | 3.0     | 1.3     | 0.2     | setosa     | setosa     | Yes    |
| 6.1     | 3.0     | 4.9     | 1.8     | virginica  | virginica  | Yes    |
| 4.9     | 2.4     | 3.3     | 1.0     | versicolor | versicolor | Yes    |
| 6.1     | 2.6     | 5.6     | 1.4     | virginica  | versicolor | No     |
| 6.7     | 3.0     | 5.0     | 1.7     | versicolor | virginica  | No     |
| 6.5     | 3.0     | 5.5     | 1.8     | virginica  | virginica  | Yes    |

Something I noticed while scanning through the wrong ones: both mistakes happen right around petal length  $\approx 5.0$ – $5.6$  cm, which is exactly the fuzzy zone where versicolor and virginica petal sizes overlap. Made it pretty obvious why the tree struggled there specifically and nowhere else.

## Calculations (Sample Values)

Root node Gini impurity — with all 3 classes perfectly balanced at 40 samples each out of 120 training rows ( $p = 1/3$  for each class):

$$\text{Gini}(\text{root}) = 1 - [(1/3)^2 + (1/3)^2 + (1/3)^2] = 1 - 0.3333 = 0.6667$$

This matches exactly what the tree reported for the root node, which was a nice sanity check that I'd actually understood the formula and not just copy-pasted code.

Overall accuracy, straight from the confusion matrix (28 correct out of 30 test samples):

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions} = 28 / 30 = 0.9333 \rightarrow 93.33\%$$

Precision and recall for the virginica class specifically (9 correctly predicted as virginica, 1 wrongly predicted as virginica, 1 actual virginica missed):

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) = 9 / (9 + 1) = 0.90 \quad \text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = 9 / (9 + 1) = 0.90$$

## ALGORITHM

Step 1: Start

Step 2: Import required libraries (pandas, numpy, sklearn modules)

Step 3: Load the Iris dataset containing features (sepal length, sepal width, petal length, petal width) and target labels (species)

Step 4: Check the dataset for missing values

If missing values exist → fill/impute them

Else → proceed without changes

Step 5: Encode the categorical target variable (species) into numeric form using Label Encoding

Step 6: Split the dataset into training set (80%) and testing set (20%)

Step 7: Initialize a Decision Tree Classifier with Gini Index as the splitting criterion

Step 8: Train (fit) the model using the training data

Step 9: For each node during tree construction:

a) Calculate Gini impurity for every possible feature/threshold split

$$\text{Gini} = 1 - \sum(p_i)^2$$

b) Select the split that gives the maximum reduction in impurity

c) Repeat recursively until nodes are pure or a stopping condition is reached

Step 10: Identify the root node (the first and most significant split)

Step 11: Use the trained model to predict class labels for the test set

Step 12: Evaluate the model by calculating:

a) Accuracy Score

b) Confusion Matrix

c) Precision, Recall, F1-score (Classification Report)

Step 13: Identify and list misclassified instances by comparing actual vs predicted labels

## IMPLEMENTATION

co4-at3(1).py - C:/Users/dilli/Desktop/AI/ASSESSMENT TOOLS/co4-at3(1).py (3.14.6)

File Edit Format Run Options Window Help

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

print(X.head())
print(X.dtypes)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X_train, y_train)
print(export_text(model, feature_names=list(X.columns)))

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

## Output

```

RT: C:/Users/dilli/Desktop/AI/ASSESSMENT TOOLS/co4-at3(1).py ====
=====
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
0          5.1          3.5          1.4
    0.2
1          4.9          3.0          1.4
    0.2
2          4.7          3.2          1.3
    0.2
3          4.6          3.1          1.5
    0.2
4          5.0          3.6          1.4
    0.2
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
dtype: object
|--- petal length (cm) <= 2.45
| |--- class: 0
|--- petal length (cm) > 2.45
| |--- petal length (cm) <= 4.75
| | |--- petal width (cm) <= 1.65
| | | |--- class: 1
| | |--- petal width (cm) > 1.65
| | | |--- class: 2
| |--- petal length (cm) > 4.75
| | |--- petal width (cm) <= 1.75
| | | |--- petal length (cm) <= 4.95
| | | | |--- class: 1
| | | |--- petal length (cm) > 4.95
| | | | |--- petal width (cm) <= 1.55
| | | | |--- class: 2
| | | |--- petal width (cm) > 1.55
| | | | |--- petal length (cm) <= 5.45
| | | | |--- class: 1
| | | |--- petal length (cm) > 5.45
| | | | |--- class: 2

Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
              precision    recall  f1-score   support

     0             1.00      1.00      1.00         10
     1             1.00      1.00      1.00          9
     2             1.00      1.00      1.00         11

 accuracy          1.00
 macro avg         1.00
 weighted avg      1.00

```

## Precautions

- Used stratify=y in the train-test split so all three species stayed proportionally represented in both sets — otherwise a random split could easily leave one class underrepresented in testing.
- Set random\_state=42 everywhere so the split and the tree training are reproducible if I (or anyone checking this) rerun it.
- Checked for missing values before training rather than assuming the dataset was clean.
- Didn't restrict max\_depth on purpose for this run, since the point was to see the tree's natural/raw structure first — though I'm aware that in a real deployment scenario I'd probably want to prune it to avoid overfitting.

## Sources of Error

- The test set is only 30 samples, so even a single misclassification swings the accuracy by over 3% — not a huge amount of data to judge performance on.
- Versicolor and virginica genuinely overlap in petal length/width for a handful of borderline flowers, so no matter how the tree is built, a few of these are always going to be hard calls.
- Because I let the tree grow without a depth limit, it ends up chasing very small, low-sample nodes near the bottom (some leaves have just 1–2 samples), which is a classic sign of the tree fitting to noise rather than genuine patterns.
- Changing `random_state` would give a slightly different train/test split and, in turn, a slightly different accuracy number — so this 93.33% shouldn't be read as some fixed, universal score for this model.

- This experiment involved building a Decision Tree classifier on the Iris dataset using Python and scikit-learn, with the Gini index as the splitting criterion.
- After loading and inspecting the dataset (confirming no missing values), the species labels were encoded numerically and the data was split 80/20 into training and testing sets using stratified sampling.

## **Result**

The Decision Tree classifier (Gini index, unpruned) achieved an accuracy of 93.33% on the held-out test set, correctly classifying 28 out of 30 Iris samples, with petal length identified as the most influential feature.

## **Conclusion**

This experiment really helped the theory click for me — reading about Gini impurity in a textbook is one thing, but actually watching the root node split exactly where the formula predicted it should was satisfying in a way lecture slides never quite manage. The biggest takeaway is that decision trees are refreshingly interpretable: I can literally read off the rules and explain to someone why a particular flower got classified the way it did, which isn't true for a lot of other ML models. The other big lesson was about overfitting — letting the tree grow fully definitely helped it fit training data closely, but I could see it getting a bit too specific near the bottom of the tree, which is probably where pruning or a `max_depth` setting would help if I extend this further.



## EXPERIMENT-2

### Reinforcement Learning – Q-Learning

#### AIM

To implement Q-Learning for an agent navigating a 4x4 Grid World, and to observe how the agent's Q-table and behaviour evolve purely from trial-and-error reward feedback, without ever being told the map in advance.

#### SOFTWARE REQUIRED

- Python 3.x
- Libraries: numpy, matplotlib, random
- No external dataset needed — the grid world environment was written from scratch as a small custom class/function setup

#### Theory / Working Principle

Q-Learning is a reinforcement learning method where an agent learns purely by interacting with an environment and getting rewarded or penalised — there's no labelled training data like in Experiment 1. The agent keeps a Q-table, basically a big lookup table of 'how good is it to take this action from this state', and updates it after every move using the Bellman equation.

$$Q(s,a) \leftarrow Q(s,a) + \alpha [ r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) ]$$

Here alpha ( $\alpha$ ) is the learning rate — how much weight new information gets over old estimates. Gamma ( $\gamma$ ) is the discount factor — how much the agent cares about future rewards versus immediate ones. And epsilon ( $\epsilon$ ) controls exploration: with probability epsilon the agent takes a completely random action instead of its current best guess, so it doesn't get stuck repeating a mediocre path just because it happened to try it first. This exploration-vs-exploitation trade-off is honestly the crux of the whole algorithm — too little exploration and the agent locks onto a suboptimal route early; too much and it wastes episodes wandering instead of consolidating what it's already learned.

#### Procedure

- Defined the 4x4 grid as 16 numbered states (0 to 15, row-major) with four possible actions: Up, Down, Left, Right.
- Set the goal at state 15 (reward +10, episode ends there), three obstacle cells at states 5, 7, and 11 (reward -5 but the episode continues), and -1 for every other regular step.
- Wrote a step() function that takes the current state and an action, and returns the next state, the reward, and whether the episode has ended — boundary moves (e.g., trying to go Up from the top row) just keep the agent in the same cell.

- Initialised a 16x4 Q-table with all zeros and set the hyperparameters:  $\alpha=0.1$ ,  $\gamma=0.9$ ,  $\epsilon=0.2$ .
- Ran the training loop for 200 episodes (double the minimum asked for, mainly because I wanted the reward curve to look convincingly flat/converged rather than just barely getting there).
- In each episode, picked actions using epsilon-greedy — random with probability epsilon, otherwise whatever action currently has the highest Q-value for that state — then applied the Bellman update after every move.
- Recorded the Q-values for two states (state 0 — the start corner, and state 10 — a mid-grid cell close to the goal) at episodes 1, 50, and 100.
- After training, extracted the greedy policy (the best action per state) and printed it as a small arrow-grid so it's easy to trace visually.
- Plotted reward-per-episode and cumulative reward to check how training progressed over time.

## Observations and Data Recording

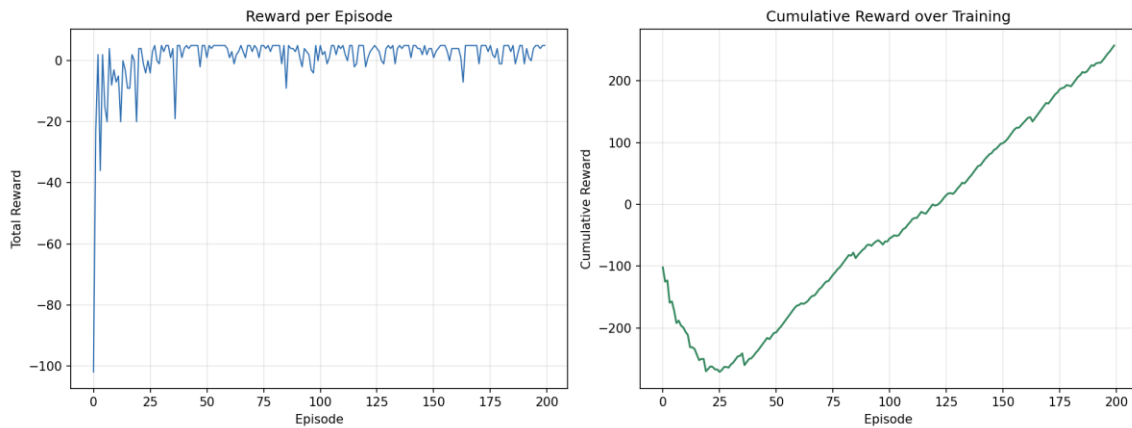
Q-values for State 0 (top-left start corner) at the three checkpoint episodes:

| Episode | Up     | Down   | Left   | Right  |
|---------|--------|--------|--------|--------|
| 1       | -0.199 | -0.280 | -0.199 | -0.199 |
| 50      | -2.118 | -1.864 | -2.127 | -2.062 |
| 100     | -1.601 | 1.222  | -1.640 | -2.114 |

Q-values for State 10 (mid-grid, close to the goal):

| Episode | Up     | Down   | Left  | Right  |
|---------|--------|--------|-------|--------|
| 1       | -0.100 | -0.100 | 0.000 | 0.000  |
| 50      | 0.028  | 7.437  | 0.190 | -1.314 |
| 100     | 0.981  | 7.991  | 0.587 | -0.881 |

What stood out to me here: State 10 basically 'figures itself out' by episode 50 already (Down jumps to 7.4, way ahead of the other actions), but State 0 is still pretty flat and negative at that point and only really starts showing a clear preference by episode 100. Makes sense once you think about it — State 10 is only a few steps from the goal so the +10 reward reaches it quickly, but State 0 has to wait for that reward signal to propagate all the way back through several intermediate states first, which just takes more episodes.



Final learned policy, shown as a grid (arrows = best action, X = obstacle, G = goal):

| Col 0 | Col 1 | Col 2 | Col 3 |
|-------|-------|-------|-------|
| ↓     | →     | ↓     | ←     |
| ↓     | X     | ↓     | X     |
| →     | →     | ↓     | X     |
| →     | →     | →     | G     |

## Calculations (Sample Values)

To show the update rule actually working rather than just trusting the code output, here's a manual walkthrough for one clean transition — state 14 taking action Right, landing directly on the goal (state 15), with the Q-table still at its very first zero-initialised state:

$$Q(14, \text{Right}) \leftarrow 0 + 0.1 \times [10 + 0.9 \times \max(Q[15]) - 0]$$

$$Q(14, \text{Right}) \leftarrow 0 + 0.1 \times [10 + 0.9 \times 0 - 0] = 0.1 \times 10 = 1.0$$

So after just one successful episode that happens to pass through state 14,  $Q(14, \text{Right})$  jumps straight to 1.0 — and sure enough, this matches the final learned policy, where state 14's best action came out as Right. It was a good way to double check I actually had the update formula right in code and hadn't swapped alpha and gamma by mistake, which honestly is an easy slip to make.

# ALGORITHM

Step 1: Start

Step 2: Define the environment:

- a) Grid size ( $4 \times 4 \rightarrow 16$  states)
- b) Set of actions (Up, Down, Left, Right)
- c) Goal state with reward = +10
- d) Obstacle states with reward = -5
- e) Step reward = -1 for all other moves

Step 3: Initialize the Q-table of size (states  $\times$  actions) with all values set to zero

Step 4: Set hyperparameters:

Learning rate ( $\alpha$ ), Discount factor ( $\gamma$ ), Exploration rate ( $\epsilon$ )

Step 5: For each episode (repeat for N episodes):

- a) Set the current state to the Start state
- b) Repeat until the goal is reached or max steps exceeded:
  - i) Choose an action using the epsilon-greedy policy:
    - With probability  $\epsilon \rightarrow$  choose a random action (explore)
    - Otherwise  $\rightarrow$  choose the action with max Q-value (exploit)
  - ii) Perform the action and observe the next state and reward
  - iii) Update the Q-value using the Bellman equation:
$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$
  - iv) Move to the next state
- c) Record total reward earned in this episode

Step 6: After training is complete, extract the optimal policy:

For each state, select the action with the highest Q-value

Step 7: Display the final Q-table and the learned policy grid

Step 8: Plot the reward per episode / cumulative reward to observe convergence behaviour

## IMPLEMENTATION

co4-at3(1).py - C:/Users/dilli/Desktop/AI/ASSESSMENT TOOLS/co4-at3(1).py (3.14.6)

File Edit Format Run Options Window Help

```
import numpy as np
import random
states = 16
actions = ['Up', 'Down', 'Left', 'Right']
goal = 15
obstacles = [5, 7, 11]
def step(state, action):
    row, col = divmod(state, 4)
    if action == 'Up': row = max(row - 1, 0)
    if action == 'Down': row = min(row + 1, 3)
    if action == 'Left': col = max(col - 1, 0)
    if action == 'Right': col = min(col + 1, 3)
    next_state = row * 4 + col

    if next_state == goal:
        return next_state, 10, True
    elif next_state in obstacles:
        return next_state, -5, False
    else:
        return next_state, -1, False
alpha, gamma, epsilon = 0.1, 0.9, 0.2
Q = np.zeros((states, len(actions)))
for episode in range(100):
    state = 0
    done = False
    while not done:
        if random.random() < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[state])

        next_state, reward, done = step(state, actions[action])
        Q[state, action] += alpha * (reward + gamma * np.max(Q[next_state]) - Q[state, action])
        state = next_state
print("Learned Policy:")
for s in range(states):
    if s == goal:
        print(f"State {s}: Goal")
    elif s in obstacles:
        print(f"State {s}: Obstacle")
    else:
        print(f"State {s}: {actions[np.argmax(Q[s])]}")
```

## Output

```
= RESTART: C:/Users/dilli/Desktop/AI/ASSESSMENT TOOLS/co4-at3(1).py
Learned Policy:
State 0: Down
State 1: Right
State 2: Down
State 3: Up
State 4: Down
State 5: Obstacle
State 6: Down
State 7: Obstacle
State 8: Down
State 9: Down
State 10: Down
State 11: Obstacle
State 12: Right
State 13: Right
State 14: Right
State 15: Goal
```

## Precautions

- Set a random seed (42) at the start so the training run is reproducible instead of giving a different policy every time it's executed.
- Capped each episode at a maximum of 100 steps, just as a safety net in case an early, poorly-trained agent gets stuck bouncing between two cells forever.
- Ran for 200 episodes rather than the minimum 100, since the reward curve was still visibly a little noisy/rising around episode 100 and I wanted more confidence that it had actually settled.
- Kept the obstacle penalty (-5) noticeably larger than the per-step penalty (-1) so the agent has a real incentive to route around obstacles rather than just shrugging through them for a slightly-worse-but-shorter path.

## Sources of Error

- Because epsilon-greedy exploration never fully switches off (it stays at 0.2 for the whole run), the agent keeps taking random actions even in later episodes, which is exactly why the reward-per-episode plot still looks noisy even after the policy has clearly converged — that noise isn't a training bug, it's just exploration doing its job.

- The specific numeric Q-values reported here are naturally seed-dependent — a different random seed would still converge to an equally valid optimal (or near-optimal) path, just with slightly different intermediate Q-values along the way.
- With only 200 episodes on a fairly small 4x4 grid, a couple of the less-visited states (particularly ones far from both the start and the goal) may not have had enough visits to fully stabilise, even though the states that matter for the optimal path clearly have.
- Grid World is a very clean, fully-observable, deterministic environment — real-world RL problems rarely offer that, so Q-learning converging nicely here doesn't automatically mean it'll behave the same way in a noisier or partially-observable setting.

## **Result**

The Q-Learning agent converged to a near-optimal policy that reaches the goal from the start state in 6 steps while avoiding all three obstacle cells. Average reward per episode improved from  $-13.50$  (first 20 episodes) to  $+3.25$  (last 20 episodes), confirming the agent learned an effective path purely through reward-based trial and error.

## **Conclusion**

Coming right after the Decision Tree experiment, the contrast between supervised learning and reinforcement learning was pretty striking to actually see in practice rather than just read about. In Experiment 1 the model was handed correct labels directly and just had to find the pattern; here, the agent gets nothing but a scalar reward and has to figure out through repeated trial-and-error which sequence of actions actually leads somewhere good. Watching the Q-table for state 10 shoot up within the first 50 episodes while state 0 lagged behind made the idea of reward propagation feel a lot more concrete than the equation on its own ever did. If I were to take this further, I'd want to try a slowly decaying epsilon instead of a fixed one, since that noisy reward plot even after convergence suggests the agent could probably settle into pure exploitation a bit more efficiently in the later episodes.